- Add LoRA to embeddings or layer normalization yet.
- Put frozen base weights into graph nodes.
- Path-sum LoRA A/B coordinates.
- Add token branches to the canvas-oriented `ModelBackend`.
- Convert existing MNIST task records into image-or-text unions.
- Add NNX.
- Make Equinox the canonical model or checkpoint representation.
- Implement a KV cache before checkpoint parity.
- Let task IDs reach any routing function.
- Evaluate routing and competence on the same token span.
- Mix JAX compilation time into steady-state latency.
- Add many command-line switches for research choices that belong in immutable experiment configurations.
- Download TinyStories or its checkpoint during ordinary unit tests.
- Preserve compatibility aliases during the generic graph refactor; the repository explicitly treats this as greenfield code.

Codex operating instruction

The implementation sequence should begin with the following directive:

> Work through `docs/LM_VAMP_EXECUTION_PLAN.md` one phase at a time. Before each phase, inspect the current graph, tests, `AGENTS.md`, `STYLE.md`, `DESIGN.md`, and `PLAN.md`. Preserve functional semantics: model parameters, optimizer state, graph state, address state, and random keys are explicit immutable values. Do not start the next phase until the current acceptance tests pass. After every phase, update `PLAN.md`; record durable architectural decisions in `DESIGN.md`; run the complete existing test suite; and report exactly which acceptance gates passed, which remain, and any measured compile/runtime behavior.

This sequence produces a validated JAX GPT-Neo implementation first, then establishes LoRA path semantics on a cheap character model, then crosses the PyTorch checkpoint boundary exactly once, and only afterward builds the three addressing methods and the full continual-learning benchmark.

Sources

Today 8:17 PM

- Here is a post mortem from codex TS experiments after executing the entire plan above.

Screening and longer prefixes

The task label is currently assigned from the entire story, but evaluation windows
are selected without checking whether the visible prefix contains that evidence. See
src/apm/data/text/curricula.py:948 and src/apm/data/text/language_tasks.py:347.

Show more

Worked for 14m 16s

Core recommendation

Do **not** try to turn the existing four-topic TinyStories curriculum into the primary scientific demonstration of VAMP. Keep it as an auxiliary **naturalistic, in-domain routing benchmark** and add the cue-stratified reporting from the post-mortem.

For the primary benchmark, build a new **knowledge-graph-first synthetic curriculum**, provisionally called **TinyWorlds-CL**:

> Construct a sequence of fictional micro-worlds as symbolic fact-and-rule graphs; derive simple TinyStories-style training narratives and precisely scored held-out queries from those graphs; then use VAMP to store, route among, compose, and eventually consolidate the LoRA competences associated with different graph regions.

This directly addresses the problems exposed by the current run:

- The current topic label is assigned from the whole story by a lexical classifier requiring a unique two-concept winner.
- Evaluation then chooses deterministic sliding spans without requiring that the selected prefix contain the labeling evidence.
- The benchmark mostly measures specialization among concepts already well represented by the frozen TinyStories model. The current results show a relatively small frozen-to-adapted NLL improvement and only about 46% exact routing on the broad topic task.

TinyWorlds should instead make the following quantities known exactly:

1. Which facts were available during each task.
2. Which facts and rules are needed to answer each query.
3. Which task or edge introduced each required fact.
4. Whether the prompt contains sufficient routing information.
5. Whether the answer was directly learned, inferred, or generalized to a new combination.
6. Which graph parent would be structurally useful.
7. Which nodes can be consolidated without losing behavior.

The approach combines the controlled skill decomposition of bAbI, the compositional relational generalization focus of CLUTRR, and the natural-language rule reasoning setup of RuleTaker, but renders the material as short, simple TinyStories-like narratives. arXiv +2 TinyStories itself establishes that synthetic, child-level narrative data can train coherent language models below ten million parameters, making this an appropriate linguistic envelope. arXiv

---

1. What the benchmark should demonstrate

I would organize the benchmark around seven explicit hypotheses.

H1. Novel knowledge acquisition

A task-specific LoRA should acquire arbitrary factual bindings that the frozen model could not have known:

- Nalo is a boat fixer.
- Nalo uses a copper hammer.
- Glimbits sleep beneath silver trees.
- Willow Bay is inside the Green Valley.

The key word is **arbitrary**. Generate the bindings deterministically only after fixing the base-checkpoint hash. Even when the relation semantics are familiar, the assignments are provably post-training.

The frozen model should be near chance on four-way candidate questions about these bindings. Independent LoRA and VAMP oracle paths should improve substantially.

H2. Exact stored competence

After later tasks are learned:

- An old node should still answer its old queries identically.
- A revision branch should not overwrite the earlier branch.
- Stored forgetting and task-free routing forgetting should remain distinct.

This is the cleanest demonstration of the addressed-memory framing: old competence remains reachable, even when a newer context contains an incompatible fact. That distinction between

retained competence, failed addressing, and lossy consolidation is central to the VAMP formulation. Addressed_Parameter_Memories

H3. Positive and negative transfer

A related parent should reduce the training required for a new task, while an unrelated parent should provide little benefit or cause negative transfer.

Measure:

$$T_j = L_{\text{root before training}}(j) - L_{\text{selected parent before training}}(j),$$

and also steps-to-threshold under:

- Root initialization
- True related parent
- Automatically selected parent
- Incorrect but lexically similar parent
- Consolidated regional parent

H4. Task-free routing

Given only a natural prefix, the three routers should recover an appropriate address:

- Exhaustive prefix NLL
- Hopfield content lookup
- Hopfield-initialized EBT refinement

Routing should be reported separately for prompts that are:

- Sufficient to identify one task
- Informative but shared among several tasks
- Deliberately ambiguous
- Cue-free negative controls

H5. Compositional generalization

The system should answer queries whose exact conclusion did not occur in training, but whose premises and rule were learned.

Examples:

- A class rule learned in task 1 plus a new instance introduced in task 3
- A relationship chain whose particular entity combination was held out
- A location hierarchy spanning an ancestor task and a child task
- A cross-branch query requiring two adapter regions

H6. Ground-truth graph recovery

Because every synthetic task has a known lineage, measure whether VAMP chooses the intended parent or at least a compatible ancestor.

This gives meaningful graph diagnostics rather than merely drawing the graph after the fact.

H7. Useful consolidation

Consolidation should demonstrate one or both of:

- Lower memory or addressing cost at bounded old-task loss
- Better transfer/generalization to a later related task

Generalization should not merely be assumed from neural parameter sharing; it should be demonstrated as a consequence of finding reusable compressed structure. That is exactly the role consolidation plays in the VAMP manuscript. Addressed_Parameter_Memories

---

2. The symbolic source of truth

The dataset should be generated from an immutable typed knowledge graph. Text is a rendering of the graph, not the authoritative representation.

2.1 Core types

A minimal symbolic schema:

Python

Run

```python
@dataclass(frozen=True)
class Entity:
    entity_id: str
    display_name: str
    entity_type: str
    introduced_by: str


@dataclass(frozen=True)
class Fact:
    fact_id: str
    subject_id: str
    predicate: str
    object_id: str
    introduced_by: str
    valid_context: str | None


@dataclass(frozen=True)
class Rule:
    rule_id: str
    premises: tuple["Literal", ...]
    conclusion: "Literal"
    introduced_by: str


@dataclass(frozen=True)
class WorldTaskSpec:
    task_id: str
    expected_parent_task_id: str | None
    task_kind: str
    introduced_entities: tuple[str, ...]
    introduced_facts: tuple[str, ...]
    introduced_rules: tuple[str, ...]
    bridge_fact_ids: tuple[str, ...]
    cue_entity_ids: tuple[str, ...]
```

Keep the first rule language deliberately small:

- Typed unary and binary predicates
- Positive Horn rules
- No unrestricted recursion
- Proof depth capped at two initially
- Context or time represented as an explicit argument, not mutable global state

This keeps the answers and proof paths mechanically verifiable.

2.2 Initial ontology

Use simple relation words compatible with TinyStories:

Entity types

- Person
- Creature
- Place
- Object
- Occupation
- Species or fictional class
- Time or context

Direct predicates

- `is_a`
- `lives_in`
- `works_as`
- `owns`
- `uses`
- `likes`
- `friend_of`
- `sibling_of`
- `parent_of`
- `located_in`
- `has_color`
- `visits`

Rule families

Occupation rule:

$$\text{works\_as}(x, r) \land \text{role\_uses}(r, t) \Rightarrow \text{uses}(x, t).$$

Class inheritance:

$$\text{is\_a}(x, c) \land \text{class\_home}(c, p) \Rightarrow \text{sleeps\_near}(x, p).$$

Location hierarchy:

$$\text{lives\_in}(x, p) \land \text{located\_in}(p, r) \Rightarrow \text{lives\_in\_region}(x, r).$$

Relational composition:

$$\text{sibling\_of}(x, y) \land \text{parent\_of}(z, y) \Rightarrow \text{parent\_of}(z, x).$$

Start with relations that have unambiguous proof semantics. Add negation, exceptions, quantities, and temporal ordering only after the basic benchmark works.

---

3. Task structure: related, branching, and revising worlds

Each task should be a **delta to a fictional micro-world**, not an unrelated broad topic bucket.

3.1 Four task kinds

Task kind	What it introduces	Expected graph behavior
`seed`	A new world family, entities, class rules, basic facts	Attach to root
`extension`	Compatible facts and instances that use inherited rules	Attach to the family seed or descendant
`revision`	Context-specific incompatible facts about existing entities	Branch from the last common valid state
`bridge`	Facts connecting two previously separate regions	Exposes single-path limits; useful for EBT or consolidation

A revision task is especially important. For example:

```
Task A:
    Nalo fixes boats with a copper hammer.
```

```
Task A-winter:
    During winter, Nalo fixes boats with a wooden mallet.
```

Queries carrying the summer or winter context should recover different facts. Sequential LoRA should tend to overwrite or blur these; VAMP should preserve both leaves.

3.2 Pilot topology

Use two families of four tasks each:

```
root
├── willow_seed
│   ├── willow_extension
│   └── willow_winter_revision
│       └── willow_bridge
└── sunny_seed
    ├── sunny_extension
    └── sunny_festival_revision
        └── sunny_bridge
```

Interleave presentation order:

```
willow_seed
sunny_seed
willow_extension
sunny_extension
willow_winter_revision
sunny_festival_revision
willow_bridge
sunny_bridge
```

That prevents family and recency from being the same signal.

For the first pilot:

- 8 tasks
- 8–12 entities per seed
- 4–6 new entities per extension/revision
- 20–30 direct facts per task
- 3–5 rules per seed or extension
- 2–4 bridge facts per bridge task
- Two or three deliberately incompatible revisions per family

Once this works, scale to:

```
4, 8, 16, 32, 64 tasks
```

with constant knowledge per task.

---

4. Novelty regimes

Every query should carry a novelty label.

4.1 Familiar schema, novel binding

The base understands the words but cannot know the assignment:

```
Nalo is the baker.
Nalo owns the blue boat.
Mira lives in Pine Hill.
```

This is the cleanest test of factual parameter memory.

4.2 Novel class

Introduce a pronounceable fictional category:

```
A glimbit is a small forest creature.
All glimbits sleep near silver trees.
Tavi is a glimbit.
```

Test:

```
Tavi became tired at night. Tavi went to sleep near the ...
```

The answer follows from a learned class rule, not a memorized entity-answer pair.

4.3 Novel role

Introduce a fictional occupation with ordinary semantics:

```
A bell-mender is someone who fixes bells.
Bell-menders use a soft wooden tool.
Miri is a bell-mender.
```

Then test a held-out instance or plot.

4.4 Contextual revision

Use the same entity and relation in two contexts:

```
In summer, Nalo uses a copper hammer.
In winter, Nalo uses a wooden mallet.
```

The correct answer depends on the addressable context.

4.5 Auditable novelty

Generate assignments from a seed derived from the frozen checkpoint and benchmark version:

$$s = H(\text{checkpoint hash} \,\|\, \text{benchmark version} \,\|\, \text{public seed}).$$

Scan proper nouns and nonce class words against the original TinyStories training text and reject occurrences. The official dataset materials state that the released TinyStories models were trained on the original training aggregate and that the V2/GPT-4 aggregate contains some original GPT-4 examples as a subset, so a cross-version audit is necessary for the current natural corpus but straightforward to avoid for newly generated benchmark bindings.

---

5. Two separate evaluation surfaces

Do not rely on continuation NLL alone.

5.1 Natural continuation evaluation

Continue reporting:

- Prefix routing
- Suffix NLL
- Perplexity
- Samples
- Stored and routed forgetting

This preserves comparability with the current language benchmark.

5.2 Exact knowledge evaluation

Add a candidate-completion benchmark.

A query is a natural TinyStories-style cloze rather than an instruction-following question:

```
Winter had come to Willow Bay. Nalo needed to fix a little boat.
He reached for his
```

Candidate completions:

```
wooden mallet.
copper hammer.
yellow brush.
small spoon.
```

The correct answer is determined by the symbolic graph. Score every candidate by normalized suffix NLL under:

- Frozen base
- Independent LoRA
- Sequential LoRA
- VAMP oracle node
- Exhaustively routed node
- Hopfield-routed node
- EBT soft address
- EBT hard-selected node
- Consolidated node

The primary metric is candidate accuracy. Also report:

$$m = L_{\text{best wrong}} - L_{\text{correct}},$$

the correct-answer NLL margin.

Candidate distractors should be difficult:

- Same semantic type
- Similar token length
- Values belonging to competing tasks
- Old values from revision branches
- Partial-inference answers for multi-hop questions

Avoid obviously mismatched distractors such as comparing a place with a tool.

---

6. Query families

Each query has an exact proof object and reasoning category.

6.1 Direct closed-book recall

The fact appeared during training, but the evaluation wording and plot are new.

```
Stored:
    Nalo owns the red boat.

Query:
    Nalo walked to the lake and looked for the boat that belonged to him.
    The boat was ...
```

6.2 Ancestor-plus-child inference

A rule was learned in an ancestor task; the child introduces the instance.

```
Ancestor:
    Boat fixers use copper hammers.

Child:
    Nalo is a boat fixer.
```

```
Query:
    Nalo began fixing a boat. He picked up his ...
```

This is a direct transfer test.

6.3 New-instance generalization

The rule is stored, but the query introduces a new temporary instance:


```
Stored:
    All glimbits sleep near silver trees.

Prompt:
    Luma had just become a glimbit. At night, Luma went to sleep near ...
```

The entity itself was never trained.

6.4 Two-hop relational composition

Use two learned facts or one fact and one rule. Hold out the exact combination.

6.5 Revision-sensitive recall

Candidates include both the old and new values. The prefix contains a context cue but not the answer.

6.6 Cross-branch composition

The proof requires facts introduced on sibling branches.

Initially, no hard VAMP path may contain both. Compare:

- Hard single-node routing
- EBT continuous edge mixture
- A later consolidated bridge node
- Symbolic KG execution

This query family exposes an actual architectural limitation instead of merely a routing error.

6.7 Open-book controls

Place all necessary premises in the prompt and ask the same cloze question.

This measures whether the base model can perform the reasoning format at all. If the open-book control fails, poor closed-book performance cannot safely be attributed to memory.

---

7. Cue visibility becomes a controlled variable

The post-mortem's three strata should become dataset fields generated by construction rather than inferred after the run.

`cue_sufficient`

The prefix uniquely identifies the intended task or context under the dataset's symbolic cue model.

Examples:

- Unique world and entity combination
- Shared entity plus unique season
- Two entities whose co-occurrence occurs in only one task

Use this as the primary exact task-routing cohort.

`cue_present`

The prefix contains relevant task-associated entities or settings, but several nodes remain possible.

Report:

- Top-k node recall
- Best-node regret
- Address entropy
- Soft-mixture performance

`cue_hidden_or_ambiguous`

The prompt intentionally lacks enough information to recover one task.

Exact task-node accuracy is not a meaningful success criterion here. Report:

- Best achievable node set
- Regret
- Calibration or entropy
- Whether EBT maintains a mixture rather than becoming arbitrarily overconfident

`cue_free_control`

Task assignment is independent of prompt contents. Routing should be at chance.

For the existing broad-topic benchmark, implement the post-mortem strata and paired 64/128/192-prefix evaluation, but explicitly label it an auxiliary natural-corpus result. The current preset uses a 256-token context, so 192-token prefixes can be paired with a 64-token suffix without changing the model.

---

8. Generate text from plans, not facts from text

The LLM generator must never determine the answer key.

8.1 Generation pipeline

```
seeded world generator
        ↓
symbolic facts, rules, task lineage
        ↓
proof closure and eligible query targets
        ↓
story plans and query plans
        ↓
template renderer or LLM surface renderer
        ↓
deterministic validation
        ↓
frozen train/validation/test artifacts
```

8.2 Stage 1: deterministic templates

Start with fully controlled renderings.

For each predicate, define several train and test templates:

```
works_as:
    "{person} worked as a {role}."
    "Every day, {person} did the work of a {role}."

uses:
    "{person} used a {tool} at work."
    "The {tool} was the thing {person} needed for the job."
```

Training and test template families must be disjoint.

This version will be less literary, but it establishes whether VAMP can store and route the knowledge.

8.3 Stage 2: constrained LLM rendering

Once the deterministic benchmark is learnable, use a generator model only to turn a symbolic plan into a child-level narrative.

The prompt should require structured output:

```json
{
  "story": "...",
  "realized_fact_ids": ["f17", "f22", "f41"],
  "sentence_fact_alignment": {
    "0": ["f17"],
    "2": ["f22", "f41"]
  }
}
```

Requirements:

- 80–180 tokens
- Simple vocabulary and sentence structure
- Every required fact realized
- No conflicting fact
- No unplanned task-specific fact
- No answer leakage into held-out prompts
- No extra named entities
- No task label or hidden ID
- Same renderer and prompt distribution across all tasks

Use a stronger model to design prompt/template families and optionally review a sample. Use a cheaper model for bulk rendering after validation is reliable. The model choice should be configuration rather than dataset semantics.

8.4 Validation

Reject a generated story unless all checks pass:

- Normalized hash uniqueness
- Required proper nouns present
- Required relation anchor words present
- Forbidden answer strings absent where applicable
- Token budget satisfied
- Entity inventory closed
- No explicit task identifier
- No contradiction with the graph
- Renderer/style distributions balanced across tasks

A second language model can flag likely semantic errors, but deterministic graph checks remain authoritative.

---

9. Symbolic splits before text splits

Split at the graph and proof level before rendering.

Training split

Contains:

- Direct descriptions of atomic facts
- Multiple surface realizations of facts
- Examples demonstrating rules
- Some but not all entity combinations

Validation split

Used for:

- Parent selection
- Content keys
- Hyperparameters
- Consolidation thresholds

Test split

Must hold out:

- Surface templates
- Plot combinations
- Query phrasings
- Selected entity pairs
- Selected proof chains
- All generated text hashes

For reasoning conclusions, the conclusion itself must not have appeared as a direct training statement. All premises may have appeared, but the exact conclusion and proof combination are held out.

---

10. Data records

I would store the benchmark as immutable JSONL plus a manifest:

```
data/tinyworlds/v1/
    manifest.json
    entities.jsonl
    facts.jsonl
    rules.jsonl
    task_specs.jsonl
    stories.train.jsonl
    stories.validation.jsonl
    stories.test.jsonl
    queries.validation.jsonl
    queries.test.jsonl
    proofs.jsonl
    atomese/
        world.metta
```

A query record:

JSON

```json
{
  "query_id": "q-willow-0042",
  "target_task_id": "willow_winter_revision",
  "router_text": "Winter had come to Willow Bay. Nalo began to fix a boat. He reached for his",
  "candidate_answers": [
    "wooden mallet.",
    "copper hammer.",
    "yellow brush.",
    "small spoon."
  ],
  "correct_candidate_index": 0,
  "required_fact_ids": ["f-role-nalo", "f-winter-tool-rule"],
  "required_task_ids": ["willow_seed", "willow_winter_revision"],
  "reasoning_type": "contextual_rule_application",
  "reasoning_depth": 2,
  "cue_regime": "cue_sufficient",
  "novelty_regime": "familiar_schema_novel_binding",
  "knowledge_mode": "closed_book",
  "surface_template_family": "winter_work_cloze"
}
```

Every query should include a machine-checkable proof.

11. Consolidation experiments

Run consolidation only after the unconsolidated benchmark has a reliable oracle ceiling.

11.1 Path folding

For a path with LoRA updates

$$\Delta W = \sum_{i=1}^{k} s A_i B_i,$$

an exact factorization is available by concatenation:

$$A_* = [A_1 \; A_2 \; \cdots \; A_k],$$

$$B_* = \begin{bmatrix} sB_1 \\ sB_2 \\ \vdots \\ sB_k \end{bmatrix}.$$

Its rank is at most kr.

Test three variants:

1. **Exact path fold:** rank kr, same function, shallower address.
2. **Truncated-SVD fold:** ranks $r, 2r, 4r$, approximate compression.
3. **Distilled rank-r fold:** train one adapter to match original-path logits and answers.

Measure:

- Maximum logit error
- Old-query accuracy
- Natural suffix NLL
- Adapter bytes
- Path depth
- Warm inference cost
- Addressing cost

The current LoRA implementation already applies independent weighted edge contributions and supports per-example continuous edge coefficients, so the functional target for folding is well defined.

11.2 Regional macro consolidation

For a compatible family:

1. Select a common ancestor and descendant region.
2. Build a balanced consolidation probe set from all member tasks.
3. Train a shared macro-adapter from the ancestor.
4. Rebase each old task as a small residual from the macro.
5. Retain the original graph version for exact comparison.
6. Prune only in a separate experimental graph after fidelity passes.

Compare:

```
original paths
macro only
macro + task residual
compressed macro + residual
```

The interesting generalization test is whether the macro:

- Performs better on unseen combinations from the family
- Reduces steps for the next related task
- Provides a better parent than any original leaf

- Reduces memory after residual rebasing

11.3 Incompatible revision control

Do not consolidate summer and winter revisions blindly.

The benchmark should contain regions where averaging is harmful. A consolidation method must either:

- Preserve separate residuals
- Maintain contextual coefficients
- Reject the merge

This prevents “consolidation” from meaning indiscriminate task-vector averaging.

11.4 Cross-branch bridge consolidation

For sibling branches whose information must be combined:

- Compare EBT soft composition before consolidation
- Create a bridge macro-node using balanced distillation
- Evaluate hard routing to the bridge node afterward

This asks whether consolidation can turn an expensive or diffuse mixture into a cheaper discrete address.

---

12. Metrics

The current language evaluator primarily stores suffix NLL, routing correctness, and routing regret. TinyWorlds should add exact symbolic competence.

Stored competence

- Four-way candidate accuracy under task node
- Correct-answer NLL margin
- Direct recall accuracy
- Accuracy by reasoning depth
- Accuracy by novelty regime
- Old-node logit drift
- Stored forgetting
- Revision consistency: old node returns old answer, new node returns new answer

Routing competence

- Exact node accuracy on `cue_sufficient`
- Top-k node recall
- Best-node regret
- Answer accuracy after routing
- Address entropy and margin
- Required-edge support recall
- Accuracy by cue regime
- Accuracy by graph size

Transfer

- Parent NLL before child training
- True-parent rank under parent search
- Steps-to-accuracy threshold
- Final child accuracy
- Root versus related-parent versus selected-parent comparison
- Consolidated-parent comparison

Graph structure

- True-parent selection accuracy

- Compatible-ancestor selection rate
- True-parent NLL rank
- Family purity of subtrees
- Depth versus latent task depth
- Bridge-task behavior

Memory

- Frozen base bytes, once
- Bytes per LoRA edge
- Total VAMP bytes
- Bytes per retained atomic fact
- Bytes per correct held-out query
- Address-key bytes
- Consolidated versus unconsolidated bytes
- Raw symbolic KG bytes as a reference point

Address cost

- Candidate nodes scored
- Prefix tokens evaluated
- Base-model forward equivalents
- Hopfield dot products
- EBT steps
- EBT support size
- Cold compile time
- Warm synchronized latency
- Cost per correct answer

Generalization

Keep these separate:

1. Surface paraphrase generalization
2. Held-out entity-combination generalization
3. Rule application to new instances
4. Deeper proof generalization
5. New-task adaptation speed
6. Cross-branch composition
7. Post-consolidation transfer

---

13. Baseline matrix

At minimum:

Method	What it isolates
Frozen base	Pre-existing competence and chance floor
Sequential LoRA	Ordinary forgetting
Independent root LoRA	Per-task storage ceiling without transfer
VAMP oracle node	Stored competence
VAMP true-parent training	Maximum available parent transfer
VAMP selected-parent training	Actual graph construction
Exhaustive routing	Accurate expensive routing
Hopfield routing	Fast approximate routing

Method	What it isolates
Hopfield → EBT	Retrieval plus refinement
Random valid node	Routing control
Wrong-family parent	Negative-transfer control
Consolidated macro	Compression and shared competence
Frozen base + oracle supporting facts in prompt	Open-book reasoning ceiling
Exact KG executor	Dataset-integrity ceiling

The exact KG executor is not a competitor to the neural system. It confirms that every answer and proof is well defined.

---

14. Integration with the existing repository

The existing implementation is already a good foundation:

- The graph is immutable and one-parent, with path incidence available.
- LoRA edges are immutable fixed-shape values.
- Training differentiates only the candidate edge and commits a new immutable node.
- The router/competence split already ensures that competence loss is inactive over the address prefix.

Do not route synthetic queries through the current arbitrary-span selector. It slides through documents and chooses hash-ordered spans. A knowledge query has an explicit semantic prefix/answer boundary.

14.1 New types

Add a parallel evaluation surface rather than overloading `LanguageEvaluationExample`:

Python Run

```python
@dataclass(frozen=True)
class KnowledgeCandidate:
    answer_text: str
    competence_batch: CompetenceBatch


@dataclass(frozen=True)
class KnowledgeQuery:
    query_id: str
    router_batch: RouterBatch
    candidates: tuple[KnowledgeCandidate, ...]
    correct_candidate_index: int
    target_task_id: TaskId
    oracle_node_ids: tuple[NodeId, ...]
    required_fact_ids: tuple[str, ...]
    required_task_ids: tuple[TaskId, ...]
    reasoning_type: str
    reasoning_depth: int
    cue_regime: str
    novelty_regime: str
    knowledge_mode: str
```

For ordinary task-local queries, `oracle_node_ids` has one member. For ambiguous or cross-branch cases it may contain several valid nodes or be accompanied by an oracle edge-support vector.

14.2 Keep current training tasks

A TinyWorlds task should contain:

```python
@dataclass(frozen=True)
class TinyWorldsTask:
    language_task: LanguageTask
    world_delta: WorldTaskSpec
    validation_queries: tuple[KnowledgeQuery, ...]
    test_queries: tuple[KnowledgeQuery, ...]
```

Existing LoRA training can consume `language_task`. The new evaluator consumes the exact queries.

14.3 Capacity changes before consolidation

The current `LanguageCurriculum` requires exactly one root plus one node per task. Before adding macro-nodes:

- Permit `max_nodes >= task_count + 1`
- Permit spare consolidation capacity
- Introduce `node_kind = task | macro | bridge`
- Record `source_task_ids`
- Allow a node with no single `trained_task`
- Keep all changes immutable

Do this only in the consolidation phase, not before the pilot.

14.4 Suggested module layout

```
src/apm/data/text/tinyworlds/
    schema.py
    ontology.py
    world_generation.py
    closure.py
    story_plans.py
    template_rendering.py
    llm_rendering.py
    validation.py
    query_generation.py
    serialization.py
    atomese_export.py

src/apm/continual/
    knowledge_tasks.py
    knowledge_evaluation.py
    knowledge_metrics.py

src/apm/lm/
    candidate_scoring.py

src/apm/memory/
    lora_consolidation.py
    regional_consolidation.py

scripts/
    generate_tinyworlds.py
    run_tinyworlds_cl.py
    consolidate_tinyworlds_graph.py

notebooks/
    tinyworlds_vamp.ipynb
```

---

15. Codex work plan

Phase 0 — Close the existing post-mortem

Retain the current TinyStories topic benchmark, but:

- Add `cue_sufficient`, `cue_present`, and `cue_hidden_or_ambiguous`

- Add paired 64/128/192 prefix evaluation
- Use a 64-token natural suffix for the 192-prefix condition
- Report the same underlying story identities at all prefix lengths
- Clearly label the benchmark as in-domain topic specialization

Gate: Existing results can be re-reported without changing training.

Phase 1 — Symbolic TinyWorlds generator

Implement:

- Typed entities, facts, rules, tasks, and proofs
- Seeded family/task lineage
- Deterministic closure engine
- Direct, one-hop, and two-hop query generation
- Same-type distractor generation
- Immutable JSONL serialization

No LLM calls yet.

Gate:

- Every query has one mechanically verified answer
- Every proof references only available facts/rules
- No split overlap
- Re-running from the same seed produces byte-identical artifacts

Phase 2 — Deterministic TinyStories rendering

Implement:

- Template-based training stories
- Disjoint train/test surface templates
- Exact cloze query prefixes and candidate suffixes
- Tokenization-aware name and distractor selection
- Prefix lengths 64/128/192
- Cue regimes generated by construction

Gate:

- All examples fit the 256-token context
- Closed-book prompts contain no answer
- Four-way candidates are type and token-length matched
- Cue-sufficient prompts have exactly one intended task under the symbolic cue checker

Phase 3 — Candidate-scoring evaluator

Implement:

- Candidate suffix NLL batching
- Hard-node answer scoring
- Routed answer scoring
- EBT soft-address answer scoring
- Correct-versus-wrong margin
- Query-type aggregation
- Proof/task-support diagnostics

Gate:

- Synthetic hand-set logits choose the expected candidate
- Wrong-node revision tests select the old answer
- Correct revision node selects the new answer
- Router prefix never includes candidate answer tokens

Phase 4 — Four-task calibration

Run one family with:

```
seed
extension
revision
bridge
```

Calibrate:

- Facts per task
- Number of paraphrases/exposures per fact
- LoRA steps
- Rank
- Candidate difficulty

Initial engineering gates, not final scientific thresholds:

- Frozen base close to four-way chance on novel bindings
- Independent LoRA clearly above chance on direct recall
- One-hop reasoning learnable
- Old-node answers unchanged after later tasks
- Revision branches preserve incompatible answers

Do not scale until this works.

Phase 5 — Eight-task pilot

Run the two-family interleaved topology.

Report:

- All existing nine language methods
- Exact knowledge metrics
- Parent-selection metrics
- Ground-truth graph comparison
- Cue-regime routing
- Direct versus reasoning accuracy
- Revision retention
- Cross-branch failures

Gate: A complete deterministic HTML/JSONL report comparable to the current language reports.

Phase 6 — LLM surface realization

Add provider-neutral rendering with:

- Prompt/version manifest
- Raw response cache
- Retry/rejection accounting
- Fact alignment output
- Deterministic validation
- Renderer-balanced task sampling

Run template and LLM-rendered versions side by side.

Gate: The LLM-rendered benchmark preserves symbolic answer validity and does not introduce task-correlated renderer artifacts.

Phase 7 — Long-stream benchmark

Scale to 16, 32, then 64 tasks.

To keep evaluation bounded:

- Run a fixed sentinel set after every task
- Run full evaluation at stages $1, 2, 4, 8, 16, 32, 64$ and final
- Measure exhaustive versus Hopfield-top-k versus EBT cost
- Interleave families and revisions

Gate: Memory, stored competence, routing competence, graph structure, and cost are all plotted against task count.

Phase 8 — Path consolidation

Implement:

- Dense effective-update materialization per projection
- Exact concatenated-rank fold
- Truncated-SVD fold
- Distilled fixed-rank fold
- Behavioral fidelity report

Gate: Exact folds match original logits; approximate folds expose explicit memory/fidelity curves.

Phase 9 — Regional macro consolidation

Implement:

- Oracle family-region selection first
- Balanced consolidation probes
- Shared macro training
- Leaf residual rebasing
- New-task transfer evaluation
- Non-destructive graph versioning

Gate: Report whether consolidation saves memory or improves transfer without hiding old-task degradation.

Phase 10 — Learned region selection

Only after oracle consolidation works:

- Cluster by adapter functional similarity
- Cluster by query behavior
- Cluster by content-key similarity
- Compare discovered clusters with latent world families

Phase 11 — AtomSpace bridge

The knowledge graph should be AtomSpace-ready from the beginning.

---

16. AtomSpace trajectory

Every TinyWorlds query should have both:

1. A natural-language cloze prompt
2. A canonical symbolic query AST

For example:

```
Natural:
    Nalo began fixing a boat. He picked up his ...

Symbolic:
    Find ?tool
    where
        works_as(Nalo, ?role)
        role_uses(?role, ?tool)
```

Export facts and rules to Atomese or MeTTa. AtomSpace's query machinery is designed to match graph patterns containing variables and return groundings, which maps naturally onto these generated query specifications. wiki.opencog.org

Develop the neural-symbolic interface in stages:

Stage A: relation prediction

Given the prompt, predict:

```
uses
lives_in
owns
parent_of
```

Stage B: entity linking and slot filling

Predict:

```
subject = Nalo
predicate = uses
object = variable
```

Stage C: query-template selection

Choose among a finite library of one-hop and two-hop query ASTs.

Stage D: full symbolic query generation

Generate a canonical serialized query and execute it against the AtomSpace.

Score separately:

- Entity-linking accuracy
- Predicate/operator accuracy
- Exact AST accuracy
- Executable-query rate
- Execution-answer accuracy
- Task-free VAMP routing
- Query-head addressing cost

The same VAMP adapters can then represent heterogeneous **views over the AtomSpace**:

- Local vocabularies
- Entity-linking conventions
- Relation families
- Query plans
- Domain-specific inference shortcuts

That is a much more direct precursor to the eventual architecture than broad topic-conditioned story generation.

Recommended first concrete experiment

Start with the deterministic eight-task pilot:

```
2 world families
4 tasks per family
24 direct facts per task
4 rules per seed/extension
1,000–1,500 training stories per task
256 validation queries per task
512 test queries per task
4 candidate answers
64/128/192-token prompt variants
8-rank LoRA edges
direct, one-hop, two-hop, revision, and bridge query classes
```

Before spending money on LLM rendering, establish:

1. Frozen base cannot answer the arbitrary bindings.
2. Independent LoRA can learn them.

3. VAMP oracle paths retain them.
4. Related parents improve learning.
5. Revision branches preserve incompatible facts.
6. Exact routing works when cues are sufficient.
7. Cross-branch queries expose a genuine hard-path limitation.
8. EBT mixture or consolidation can reduce that limitation.

That experiment would give you a substantially stronger VAMP proof than the current topic run: novel competence, known proof structure, known task lineage, exact recall tests, controllable routing ambiguity, measurable transfer, and a principled path to graph consolidation and AtomSpace query generation.

Sources